Evidence-Grounded Cybersecurity Question Answering with MITRE ATT&CK

Author: Danial Baledi

Project: MITRE ATT&CK Chatbot

Contact: baledi.danial@gmail.com

Profiles: GitHub | LinkedIn | Hugging Face

Source Code: BDanial/mitre_attack_chatbot

Contents

1. Overview and Problem Definition
2. Data Acquisition and ATT&CK Graph Extraction
3. Relational Storage in PostgreSQL
4. Structure-Aware Chunking and Metadata
5. Dense and Sparse Storage in Qdrant
6. Advanced Retrieval System
7. Agent Formulation and Orchestration
8. Testing, Evaluation, Challenges, Limitations, and Improvements
9. Disclosure of AI Assistance

1. Overview and Problem Definition

This project introduces an evidence-grounded, multi-agent Retrieval-Augmented Generation (RAG) system tailored for cybersecurity question answering. The system is designed to assist security analysts who have observed specific cyber behaviors or incident traces but require authoritative insights regarding their classification, mitigation, and handling procedures. Furthermore, it serves as a specialized encyclopedic chatbot, generating highly accurate, grounded responses derived strictly from trusted threat intelligence.

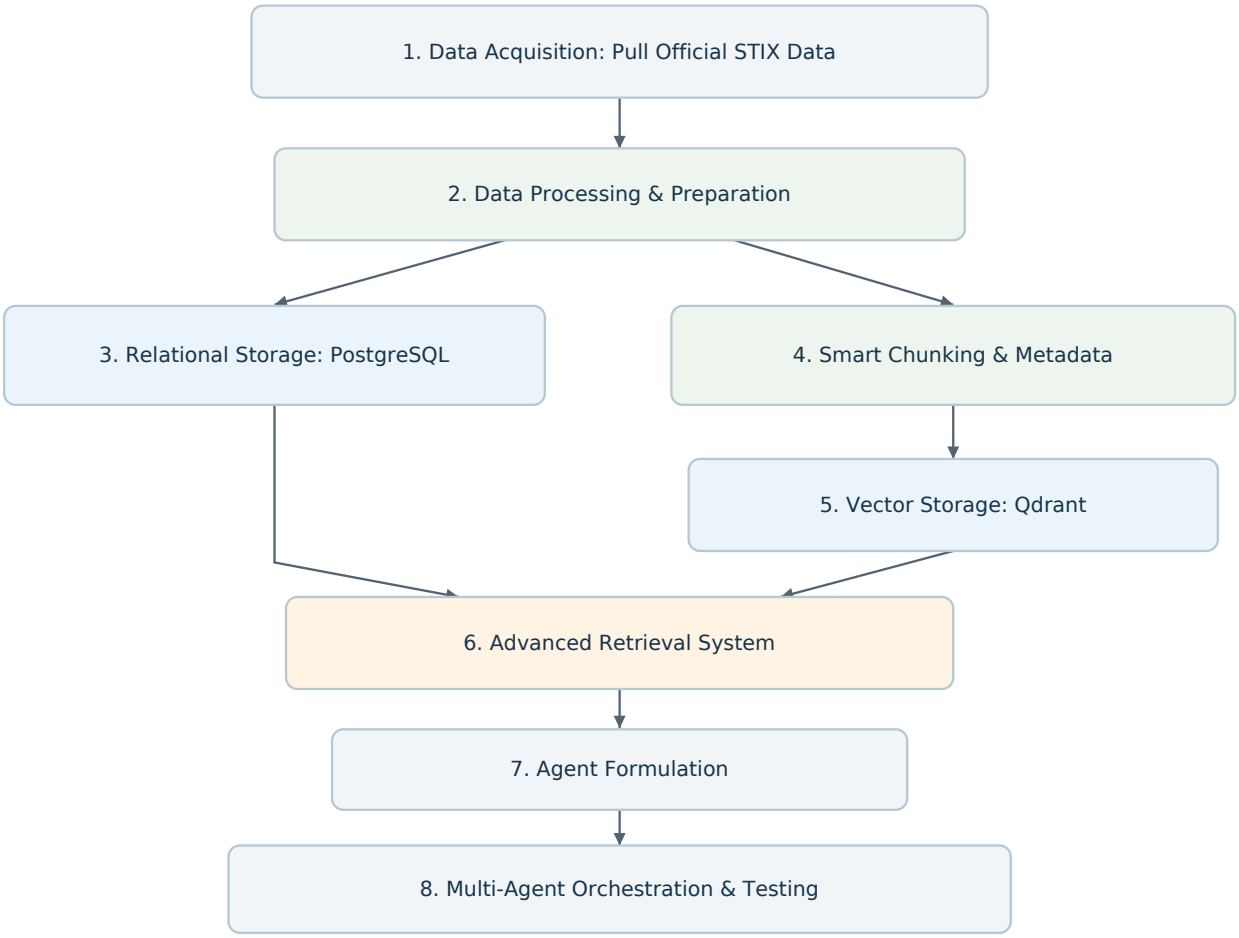
Given a user query or a short incident description, the system retrieves relevant tactics, techniques, procedure examples, mitigation guidance, and detection strategies exclusively from the **MITRE Enterprise ATT&CK framework**. The system enforces a strict answering policy: all factual claims must be supported by retrieved evidence with explicit source citations. If the retrieved evidence is insufficient, the system is instructed to explicitly state this limitation or request clarification. The intended scope is focused on ATT&CK-based investigation support rather than unrestricted, generic cybersecurity advice.

System Workflow

The development of this system was executed in eight systematic phases, designed to ensure data integrity, robust retrieval, and dynamic agent orchestration:

1. **Data Acquisition:** Pulling official ATT&CK STIX data (Ver. 19.2) and preserving both textual evidence and graph relationships.
2. **Data Processing:** Cleaning, filtering, and preparing the extracted knowledge for storage.
3. **Relational Storage:** Designing a structured schema and storing the relational entity data within a PostgreSQL database.
4. **Data Chunking:** Implementing a smart chunking strategy enriched with high-value metadata to facilitate advanced, context-aware retrieval and enhanced search flexibility.
5. **Vector Storage:** Generating dense and sparse embeddings for semantic and lexical search, and storing the resulting vectors in Qdrant.
6. **Advanced Retrieval Design:** Implementing a complex retrieval mechanism that includes Text-to-SQL for structured queries, hybrid vector search for unstructured data, and combined approaches.
7. **Agent Formulation:** Designing independent, specialized AI agents with distinct roles for processing queries, retrieving data, and synthesizing answers.
8. **Orchestration & Testing:** Integrating the agents into a cohesive multi-agent workflow and conducting rigorous testing against diverse queries.

Figure 1 illustrates the overall system architecture and workflow.



2. Data Acquisition and ATT&CK Graph Extraction

The knowledge base was obtained from MITRE's official ATT&CK STIX repository, specifically the complete Enterprise ATT&CK v19.2 (STIX 2.1 format) bundle. Although the assignment refers to preparing a small set of authoritative cybersecurity documents, this implementation deliberately does not use a small document set. It processes the full Enterprise ATT&CK bundle and retains 6,520 filtered STIX objects, from which 23,240 retrieval points are produced. STIX was selected because it provides both the descriptive content needed for retrieval and the object identifiers and relationships needed to preserve ATT&CK semantics.

The extraction stage first identifies every attack-pattern and maps its STIX identifier to its human-readable ATT&CK identifier (for example, T1059.001). Before reducing the graph, it extracts non-empty descriptions from uses relationships whose target is a technique. These descriptions are valuable procedure examples: they express how an actor, campaign, malware, or tool used a technique in an observed context. Each extracted example retains the target technique identifier, so it can later be retrieved as text and verified against the graph.

The reduced graph removes intrusion-set, campaign, malware, and tool nodes, along with relationships incident to them. This keeps the project focused on technique, mitigation, detection, tactic, analytic, data-component, and matrix knowledge while avoiding entity nodes that are unnecessary for the intended use case. The procedure text is preserved before this reduction; therefore, removal of actor/software nodes does not discard the behavioral evidence they contributed. The stage writes two derived artefacts: a filtered STIX graph and a behavior example

collection. These files form the controlled handoff to the relational-loading and retrieval-document stages.

To quantify the impact of this filtering process on the overall dataset, the following statistics were recorded during the data processing stage:

Metric	Before Filtering	After Filtering	Reduction / Status
Nodes	4,824	3,749	1,075 (22.28%)
Relationships	21,262	2,771	18,491 (86.97%)
Total STIX Objects	26,086	6,520	19,566
File Size	45.88 MiB	16.06 MiB	~65%
Extracted Behavior Examples	N/A	17,136	**Preserved**

The deleted nodes belong exclusively to the following four categories:

- **Malware:** 733
- **Intrusion Set:** 191
- **Tool:** 95
- **Campaign:** 56

As previously noted, the behavioral evidence is systematically extracted prior to node deletion. Consequently, despite the massive 86.97% drop in graph relationships (which is expected, as all relationships connected to these four node types are removed), the 17,136 valuable uses behavioral descriptions are fully preserved.

However, it is worth noting that if a comprehensive or commercial version of this application is developed for a production environment in the future, strategies will be implemented to retain these currently excluded items in some capacity, thereby offering a more exhaustive and interconnected threat intelligence database.

3. Relational Storage in PostgreSQL

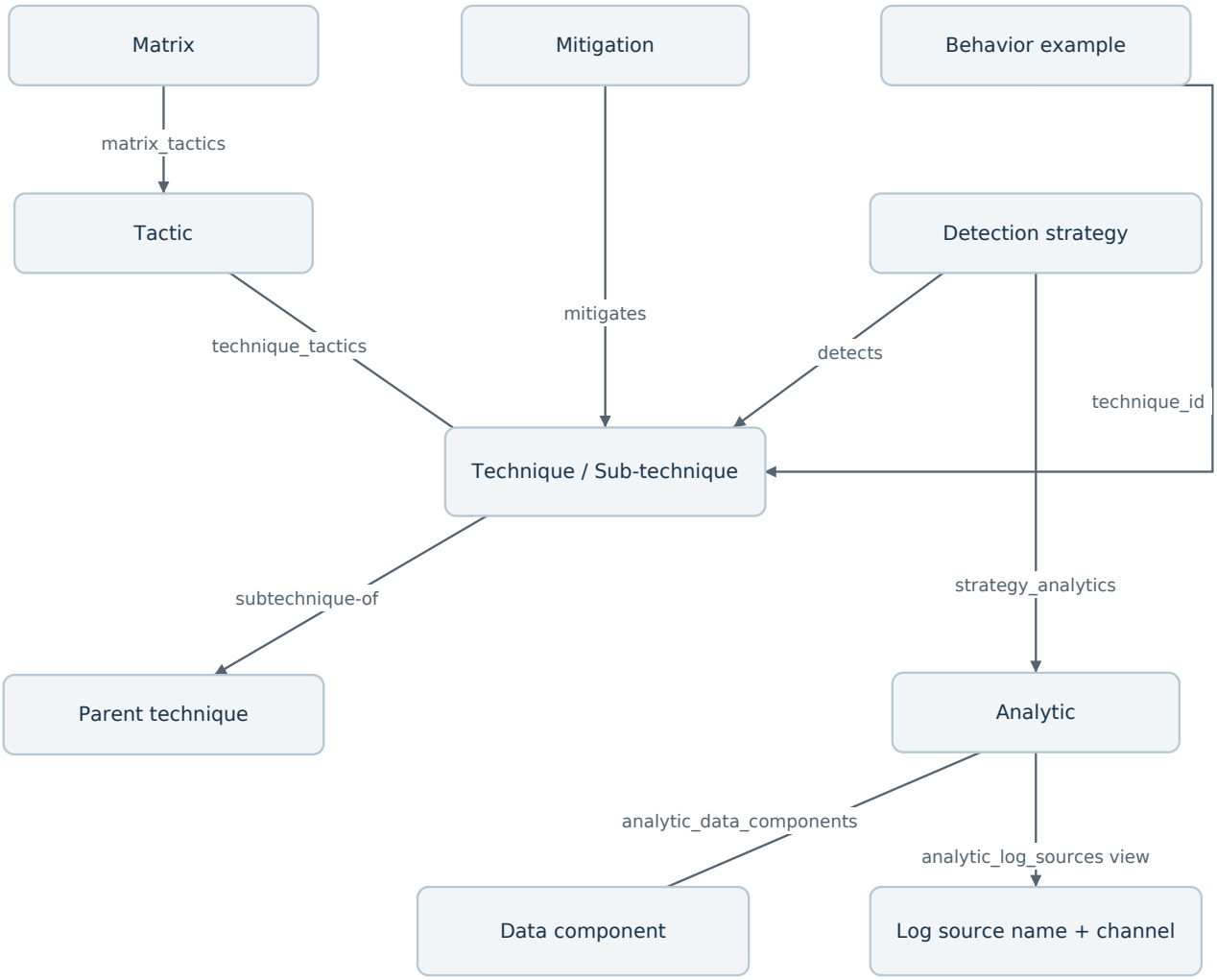
PostgreSQL is the authoritative structured store because several cybersecurity questions require exact identifiers, exhaustive lists, or multi-hop ATT&CK relationships that vector similarity cannot reliably establish. The database uses a dedicated attack schema with seven physical tables and type-specific SQL views. attack.nodes stores each retained STIX entity once, using its STIX identifier as the primary key; attack.relationships stores explicit directed STIX relationships with foreign keys to their source and target nodes. Human-readable ATT&CK IDs, names, descriptions, status flags, and the original type-specific STIX fields are preserved. The original STIX object is retained in a JSONB column, while frequently queried fields are extracted into typed columns and indexed.

Four normalized junction tables represent high-value ATT&CK paths: technique–tactic, detection-strategy–analytic, analytic–data-component, and matrix–tactic position. A separate behavior_examples table links every extracted procedure example to a technique. This model avoids duplicating nodes, supports relationship traversal with explicit join keys, and makes the following evidence paths queryable: technique-to-tactic membership, mitigation-to-technique, detection

strategy-to-technique, and detection strategy-to-analytic-to-data component. Views such as attack.techniques, attack.mitigations, and attack.analytics provide readable, type-safe query targets without duplicating the node data.

The import is transactional. The importer creates the schema and constraints, takes a PostgreSQL advisory transaction lock to serialize concurrent imports, clears only importer-owned tables, and loads parent tables before child tables using COPY FROM STDIN. A failed load rolls back the complete refresh, preventing a partially updated ATT&CK graph from being exposed. Primary keys, foreign keys, type checks, unique composite keys, and indexes on ATT&CK IDs and relationship endpoints protect referential integrity and common graph traversals. For the reviewed snapshot, the relational store contains 3,749 nodes, 2,771 explicit relationships, 17,136 behavior examples, and 7,033 normalized junction rows.

Figure 2 illustrates the core database relationships and entity connections mapped within the attack schema. For a comprehensive breakdown of the PostgreSQL schema design and table structures, please refer to Appendix A.



4. Structure-Aware Chunking and Metadata

The retrieval corpus was built from evidence units rather than by splitting one large document at a fixed character count. Each unit has a source-specific construction rule. Technique and mitigation points use their own descriptions; behavior examples retain the extracted uses description; relationship-specific mitigation guidance is stored separately; analytics include their detection text and linked strategy context. Detection-strategy descriptions are empty in the reviewed ATT&CK snapshot, so the pipeline uses the descriptions of their linked analytics as separate, explicitly labelled segments instead of inventing a summary. Sub-techniques receive a parent-prefixed title, and analytics receive linked strategy names as context. Thus, the text given to the embedding model preserves the meaning of the source row and the relevant ATT&CK relationship.

After cleaning Markdown, HTML, citation markup, and redundant whitespace, the pipeline splits each source segment with a conservative 3,000-byte UTF-8 budget. It first preserves sentence boundaries, then falls back to word boundaries, and only hard-splits a pathological uninterrupted string. Commands, paths, inline code, event identifiers, visible link names, and technical URLs are retained. Each dense-model input is formatted strictly, and is rejected if it exceeds 7,000 UTF-8 bytes. This policy is structure-aware because it begins from the semantic unit and its source role, adds only verifiable graph context, and uses length-based splitting only as a final constraint. It avoids mixing unrelated ATT&CK objects in arbitrary sliding windows and does not create LLM-generated summaries.

Every Qdrant point carries the chunk plus metadata logically organized into five groups to support safe filtering, source citation, SQL verification, and index diagnosis:

- **Provenance:** Identifies the source through db_schema, db_table, db_id, document_id, source_urls, title, text, and text_origin.
- **Chunk Integrity:** Records chunk_index, chunk_count, embedding_text, and content_hash.
- **Versioning:** Records dataset_snapshot, ATT&CK dataset_versions, pipeline_version, embedding_model, and embedding_dimensions to make results traceable to one reproducible corpus and vector space. Context fields (language, domain, STIX type, timestamps) are also preserved.
- **Security Context:** Records type, STIX/ATT&CK IDs and names, related tactics, platforms, parent technique fields, and status fields including is_active, revoked, deprecated, and active_technique_ids.
- **Type-Specific Metadata:** Records analytic IDs, strategy IDs, data-component IDs, log sources, and mitigation endpoints where applicable.

5. Dense and Sparse Storage in Qdrant

Qdrant stores one point per evidence chunk; a PostgreSQL row can therefore produce several points without losing the back-reference to the original row. The reviewed index contains 23,240 points across six evidence types: techniques/sub-techniques, behavior examples, mitigations, mitigation-to-technique relationships, detection strategies, and analytics. Tactics, matrices, and data components remain available through PostgreSQL joins but are not embedded as standalone points. This focuses the vector corpus on text that can directly support an answer.

Each point has an unnamed dense vector generated with google/gemini-embedding-2 through the OpenRouter embeddings API. The vector is 3,072-dimensional, stored on disk, and searched with cosine distance. Dense retrieval is appropriate when an incident description and ATT&CK evidence

have similar meaning but do not share exact wording. Deterministic UUIDs are derived from the snapshot, vector profile, source document, chunk position, and content hash, allowing a failed indexing job to resume without duplicating points.

The same point also receives the named sparse vector lexical_bm25_v1. Qdrant server-side BM25 is constructed from the human-readable title and text, using word tokenization, lowercase matching, English Snowball stemming, stop-word removal, and a pinned BM25 profile (k=1.2, b=0.75). Sparse retrieval is useful for ATT&CK IDs, commands, tool names, paths, and other precise security terminology that dense similarity can blur. Qdrant payload indexes are created for evidence type, source identity, snapshot, technique IDs, platforms, and active status to support filtered retrieval.

Embedding Text Formatting

To maximize semantic representation, the exact string passed to the embedding model is formatted strictly as title: {title or 'none'} | text: {cleaned chunk} and stored in the embedding_text payload. The formulation logic relies on the evidence type:

Evidence Type	Title Field Content	Body / Chunk Content
Technique	Technique name (Parent name prefixed if sub-technique)	Technique description
Behavior Example	none (stored as JSON null in payload title)	Cleaned behavior/procedure description only
Mitigation	Mitigation name	General mitigation description
Detection Strategy	Strategy name	Strategy description (or linked analytic descriptions)
Analytic	Linked strategy name(s) + Analytic name	Analytic detection description
Mitigates Relation	Mitigation name + "mitigates" + Technique name	Relationship-specific guidance

For a comprehensive breakdown of the QDrant schema implementation, please refer to Appendix B.

6. Advanced Retrieval System

The retrieval layer exposes two authenticated, LLM-callable tools through FastAPI. These tools enable the agentic workflow to gather both structured relational data and unstructured semantic evidence safely and efficiently.

6.1. SQL Validation Tool (query_sql)

Used by the Dify workflow orchestrator, this tool accepts a single PostgreSQL SELECT statement or a read-only Common Table Expression (CTE). It enables the model to ask exact questions that are unsuitable for vector retrieval, such as looking up an ATT&CK ID, traversing multi-hop mitigation/detection paths, obtaining exhaustive lists, or verifying if a retrieved relationship is active.

Security Constraints: The SQL parser strictly rejects multiple statements, writable CTEs, SELECT INTO, and row-locking clauses. Execution is bound by a read-only transaction, a qualified attack schema, a 5-second statement timeout, and a hard limit of 500 rows. In a production environment, this is paired with a least-privilege database role to ensure robust isolation.

6.2. Evidence Search Tool (search_attack)

This tool accesses the Qdrant vector database and offers three distinct retrieval modes dynamically selected by the LLM:

1. **Semantic:** For behavioral or conceptual similarity.
2. **Lexical:** For exact English keywords, commands, and precise technical vocabulary.
3. **Hybrid:** Executes both dense cosine and sparse BM25 retrieval against the same filtered corpus.

Reciprocal-Rank Fusion (RRF): In hybrid mode, the backend applies weighted RRF. The LLM supplies relative weights (e.g., semantic 0.7, lexical 0.3); the service normalizes these and fuses the ranks rather than attempting to mix mathematically incompatible cosine and BM25 score scales. For non-English queries, the model can simultaneously provide a translated English lexical_query to ensure exact terminology matching.

Typed Filtering: To prevent the LLM from hallucinating complex database syntaxes, the tool provides simple, typed filters (e.g., evidence types, related platforms, is_active status) combined via logical AND/OR operators.

6.3. Sequential Execution

The LLM can sequence these tools organically. For example, it can run a hybrid search over an incident description to discover candidate techniques, and then route the returned ATT&CK IDs to the query_sql tool to validate tactic membership or mitigations. The final agent synthesizes its answer based strictly on the returned rows and evidence payloads, citing source identifiers, or explicitly requesting clarification if the retrieved evidence is insufficient.

7. Agent Formulation and Orchestration

Dify was utilized as the agentic backend, defining a multi-agent system with three distinct roles. The agents exchange compact, strongly-typed structured information (e.g., column names, rows, ranked chunks, normalized weights, and metadata) rather than unstructured hidden states, ensuring deterministic data flow.

7.1. Agent Roles and Boundaries

Agent Design Principle: Effective agentic design depends on well-defined responsibilities, efficient coordination, and alignment with the capabilities of the underlying language model. The number of agents is not itself a measure of architectural quality. Each additional agent should serve a distinct purpose that justifies its communication overhead, latency, and added failure modes. Agent boundaries, tool interfaces, and context allocation should therefore reflect the primary model’s ability to follow instructions, select tools, and use returned evidence. In this project, the compact three-role architecture keeps orchestration centralized and database-facing workflows narrowly scoped; further decomposition should be motivated by a demonstrated need rather than agent count alone.

- **Text-to-SQL Agent (text_to_sql):** Receives an LLM-generated read-only SQL statement and limit, then invokes the authenticated /tools/sql endpoint.

- **Search Agent (search_qdrant):** Receives a structured search request and invokes the /tools/search endpoint.

These two workflow agents possess intentionally narrow execution boundaries: they do not receive direct database credentials and do not construct raw queries. Their DSL definitions provide reproducible configurations (available in dify_dsls/mitre_text_to_sql.yml and dify_dsls/mitre_search_qdrant.yml).

7.2. Orchestrator Agent (ReAct Strategy)

The primary orchestrator is the Dify framework. It receives the user question and conversation context, deciding whether the task requires structured SQL evidence, vector search, or a sequential combination of both.

The orchestrator employs ReAct: Synergizing Reasoning and Acting in Language Models (Yao, et al. 2022) strategy. A memory window of 25 turns allows for contextual follow-up questions while strictly enforcing the policy that all responses must be grounded in currently retrieved evidence (configured in dify_dsls/mitr_attack.yml).

To see the full orcherastor system prompt, please refer to docs/orchestrator/system-prompt.md.

7.3. Inter-Agent Communication and Resilience

System resilience is a core component of the orchestration policy. The workflow agents are equipped with fail branches and retry mechanisms. If retrieval remains unavailable or yields no evidence, the agent is strictly instructed to explicitly state the limitation rather than fabricate a response.

7.4. Technology and Model Selection Rationale

The rationale for most architectural and technology choices has already been presented in the preceding sections. The following discussion therefore focuses on several selections whose justification was not previously stated directly.

Language Model: The language model is configurable independently of the agent architecture and retrieval backend. The current deployment uses GPT-5.6 (openai/gpt-5.6-sol through OpenRouter) because it provides a capable general-purpose baseline across instruction following, tool use, reasoning, structured outputs, and multi-turn interaction. The workflow does not depend on capabilities unique to this model: the model can be replaced with a smaller or larger model, or with an open-source or commercial model, provided that the replacement can follow the tool contracts and grounding policy reliably. This separation allows deployments to choose a different balance of quality, latency, privacy, infrastructure requirements, and cost without redesigning the agents or databases.

Embedding Model: google/gemini-embedding-2 was selected based on the author’s practical experience with embedding models. Among the models considered for this project, it provided the strongest combination of multilingual representation and support for varied retrieval tasks. These properties are particularly relevant because users may describe security behavior in languages such as Persian while the ATT&CK evidence is predominantly written in English. The same model is used consistently for document and query vectors, while lexical BM25 retrieval remains available when exact terminology, identifiers, commands, or paths are more important than semantic similarity.

Vector Database: Qdrant was selected because it is among the fastest-performing vector databases in the evaluations considered during the project and provides substantial flexibility in both storage and retrieval. It supports dense vectors, native sparse BM25 vectors, cosine similarity, payload metadata, indexed filters, hybrid retrieval, and collection aliases within the same system.

These capabilities allow one evidence point to carry both retrieval representations and a verifiable link to its PostgreSQL source. They also support filtered search, resumable indexing, and atomic alias-based publication without requiring separate dense and lexical databases.

Agent Framework: Dify was selected because it provides a production-oriented environment for assembling and operating agentic applications, rather than only a low-level experimental agent library. Its visual workflows, reusable tools, configurable model providers, conversation memory, credential handling, retry and failure branches, and exportable DSL definitions made it faster and more reliable to turn the retrieval backend into an operational chatbot. Dify also keeps the model, tools, and orchestration configuration separable, which simplifies maintenance and future model replacement.

Structured and Hybrid Retrieval: PostgreSQL remains the authoritative structured store because exact ATT&CK identifiers, complete lists, and multi-hop graph relationships require deterministic queries and cannot be guaranteed by similarity search. Qdrant complements it with semantic and lexical discovery over evidence text. Weighted reciprocal-rank fusion is used for hybrid retrieval because dense cosine scores and BM25 scores are not directly comparable. The orchestrator can select either retrieval path or combine them dynamically, giving the system both exploratory semantic recall and structured verification.

7.5. Technology Stack

The following table summarizes the principal technologies used across the project. The deployment and development tooling is representative rather than exhaustive because supporting tools varied across local development, hosting, testing, and service troubleshooting.

Area	Technologies	Role in the Project
Source Data and Formats	MITRE Enterprise ATT&CK 19.2, STIX 2.1, JSON, Markdown	Authoritative cybersecurity data, graph relationships, intermediate artefacts, and documentation.
Core Language and Packaging	Python 3.10+, uv, Hatchling	Backend implementation, dependency management, command-line tooling, and package builds.
Document Processing	Python, markdown-it-py	Cleaning ATT&CK text, preserving technical content, constructing evidence units, and structure-aware chunking.
Relational Storage	Self-hosted PostgreSQL, psycopg, pglast	Authoritative entity and relationship storage, transactional imports, read-only query execution, and SQL validation. PostgreSQL was self-hosted after the earlier Neon deployment produced operational errors.
Embeddings	google/gemini-embedding-2, OpenRouter, OpenAI-compatible Python client	Multilingual and multitask dense embedding generation for documents and queries.
Vector and Lexical Storage	Qdrant, qdrant-client, native BM25	Dense semantic search, sparse lexical search, metadata filtering, hybrid retrieval, and index publication.

Area	Technologies	Role in the Project
Retrieval and Fusion	Text-to-SQL, cosine similarity, BM25, weighted reciprocal-rank fusion	Exact graph lookup, semantic discovery, precise term matching, and combined ranking.
API Backend	FastAPI, Uvicorn, Pydantic models, API-key authentication	Authenticated SQL and evidence-search tools exposed to the agent workflows through HTTP and OpenAPI.
Agentic AI	Dify, GPT-5.6, ReAct, importable Dify DSL workflows	Dynamic tool selection, multi-turn orchestration, error recovery, evidence-grounded synthesis, and source-aware responses.
Frontend	HTML5, CSS3, embedded Dify chat interface	Public project presentation and browser-based access to the chatbot.
Testing and Code Quality	pytest, Ruff, synthetic fixtures, mocked model responses, local Qdrant tests	Offline verification of data processing, API behavior, SQL safeguards, retrieval contracts, and indexing logic.
Deployment and Operations	Docker, Nginx, Cloudflare client/tunnel, hosted PostgreSQL, hosted Qdrant, HTTPS reverse proxy, environment-based secrets	Service packaging, routing, secure external access, database hosting, configuration, and operational availability.
Version Control and Documentation	Git, Markdown, Mermaid	Change tracking, reproducible documentation, and architecture diagrams.

8. Testing, Evaluation, Challenges, Limitations, and Improvements

8.1. Evaluation Design and Results

To evaluate the system’s accuracy and adherence to the evidence-grounded policy, a manual validation test was conducted. Five pages were selected at random from the official MITRE Enterprise ATT&CK website. General-purpose chatbots were asked to make the random selection under the restriction that malware, campaign, and other entity categories outside the scope of this chatbot must not be selected. Two distinct questions were formulated from each selected page, producing a test set of 10 questions.

For each page, the two questions were asked as a multi-turn conversation containing two to four turns. The experiment was also repeated in a single-turn setting. In both settings, all 10 questions were answered correctly and completely. The evaluation explicitly checked two separate aspects of every response: the factual correctness and completeness of the answer, and the correctness of the precise supporting source identified by the chatbot. Both checks passed for all 10 questions. The evaluation questions and their corresponding answers are available as CSV files in the evaluation folder.

In addition, 10 boundary-handling questions covering irrelevant, ambiguous, and out-of-scope inputs were tested. Irrelevant and out-of-scope requests were correctly redirected to the chatbot’s primary ATT&CK-focused scope, while ambiguous requests were handled without presenting an unsupported interpretation as fact. All 10 boundary-handling tests passed.

Accordingly, every criterion used in this experiment received a full score: factual answer correctness, answer completeness, precise supporting-source correctness, multi-turn performance, single-turn performance, and safe handling of irrelevant, ambiguous, and out-of-scope inputs each achieved 10/10 (100%). These results apply to the defined 10-question in-scope set and the 10 boundary-handling questions.

8.2. Challenges

One of the principal technical challenges was processing the ATT&CK graph and reducing it to the scope required by the project without losing the behavioral evidence needed for retrieval. Malware, campaign, intrusion-set, and tool nodes had to be removed while useful procedure descriptions from their uses relationships were preserved and linked back to the relevant techniques. The resulting graph also had to be represented consistently across PostgreSQL, Qdrant, and the metadata used for source attribution.

The breadth of the implementation was another major challenge. The project required coordinated work across the supporting backend, database design, document processing, embedding and retrieval pipelines, agentic AI workflows, deployment, and DevOps. Bringing up and maintaining PostgreSQL, Qdrant, the authenticated FastAPI services, external embedding access, and the Dify workflows as one operational system required careful configuration, service monitoring, error handling, and consistency between components. Keeping these services available throughout ingestion, indexing, retrieval, and chatbot testing was therefore an important engineering challenge in addition to the AI design itself.

8.3. Limitations

The evaluation was a focused manual assessment based on 10 in-scope questions and 10 boundary-handling questions. Although every evaluated criterion achieved a full score, the sample is too small to establish general performance across the full range of possible ATT&CK questions. The evaluation did not include systematic prompt-injection or jailbreak testing, large-scale automated benchmarking, or extensive simulation of infrastructure and third-party service failures. Metrics such as answer relevance, faithfulness, context recall, and context precision were not calculated using an automated RAG evaluation framework.

The knowledge base is limited to the Enterprise ATT&CK domain and deliberately removes structured malware, campaign, intrusion-set, and tool entities. Their useful behavioral descriptions are preserved, but the system cannot provide complete structured attribution for those excluded entity types. Semantic and hybrid retrieval also depend on an external embedding provider, so provider availability, latency, and rate limits can affect those retrieval modes. These constraints define the scope within which the reported evaluation results should be interpreted.

8.4. Failure Cases

During backend and integration testing, transient service and resource errors were encountered, including HTTP 507 and HTTP 503 responses. These issues occurred in the supporting infrastructure and retrieval pipeline rather than in the chatbot’s reasoning behavior. They were addressed during backend testing through service recovery, retrying interrupted operations, and completing the affected indexing or retrieval workflows after the underlying service became available. The resumable indexing design reduced the need to repeat already completed work.

No answer-generation, source-identification, multi-turn, or boundary-handling failure was observed in the evaluated chatbot test set. This does not imply that the system is failure-free; it means only that no behavioral failure was observed among the scenarios included in the current manual evaluation.

8.5. Possible Improvements

Future evaluation should use a larger and more diverse question set, including more complex multi-turn conversations, multilingual inputs, difficult ambiguous cases, and systematic adversarial tests such as prompt injection and jailbreak attempts. Automated RAG evaluation could be added to measure answer relevance, faithfulness, context recall, and context precision at scale. Controlled failure-injection tests could also evaluate behavior during database outages, embedding-provider failures, timeouts, partial retrieval results, and inconsistent service states.

The knowledge base could be expanded to retain structured malware, campaign, intrusion-set, and tool information, enabling richer attribution and relationship analysis. Operational improvements could include stronger monitoring, automated health checks, alerting, capacity planning, provider fallback strategies, and more extensive end-to-end regression testing across PostgreSQL, Qdrant, FastAPI, and Dify.

A further extension would connect the agentic system to authorized security-management servers so that it could translate ATT&CK-grounded analysis into proposed defensive rules or configurations. For example, the system could map an observed behavior to the most relevant ATT&CK technique and then draft an IAM, WAF, SIEM, EDR, or application-security rule, such as requiring a CAPTCHA or step-up verification after three consecutive failed password attempts. Such a capability should initially operate in recommendation or dry-run mode: generated rules must be validated against the target platform, reviewed and approved by an authorized operator, recorded in an audit trail, tested for unintended effects, and deployed with a rollback mechanism. This would extend the project from evidence-grounded question answering toward controlled, evidence-grounded defensive action without allowing the model to make unreviewed production changes.

9. Disclosure of AI Assistance

AI tools were used to varying degrees during the design, implementation, deployment, and documentation of this project. The table below provides approximate, self-reported estimates of the extent of AI assistance within each work area. These estimates are not measured proportions of AI-authored code or text, and they should not be combined into an overall project percentage. They describe assistance during project development, rather than the use of AI within the application’s functionality.

Work Area	Scope / Technologies	Estimated AI Assistance
System Design	System architecture and design	10%
DevOps and Hosting	Deployment and hosting configuration	20%
AI and Agentic Backend	Dify-based agent workflows and orchestration	5%
Supporting Backend	Python, FastAPI, psycopg, pglast, Qdrant integration, and related backend code	70%
Frontend	HTML interface implementation	90%
Documentation	Preparation of project documentation	50%